

Activity 4: Application Architecture Design for IT Asset Management

Activity Overview

Activity Title: Application Architecture Design Using the C4 Model

Objective: You will design the application architecture for their fictional company's IT asset management transformation using the C4 model (Context, Container, Component, and Code levels). This activity builds on the TO-BE business processes from Activity 2 and the data architecture from Activity 3, translating them into a structured, multi-level application design that will guide implementation.

Why the C4 Model for Application Architecture?

The C4 model, created by Simon Brown, provides a hierarchical approach to visualizing software architecture. It addresses a fundamental challenge: different stakeholders need different levels of detail. A business executive doesn't need to see your class diagrams. A developer doesn't need to see the system context diagram. The C4 model provides four levels of abstraction, each serving a specific audience and purpose.

Level	Name	Audience	Abstraction	Purpose
Level 1	Context	Everyone	System boundaries	Show what system does and who uses it
Level 2	Container	Technical & Non-technical	Technology building blocks	Show high-level technology choices

Level	Name	Audience	Abstraction	Purpose
Level 3	Component	Developers	Internal structure	Show modular decomposition
Level 4	Code	Developers	Detailed implementation	Show critical components

For the PUP IT Asset Management transformation, the C4 model will document the architecture of the Unified Integration & Intelligence Layer, the Polyglot Persistence Layer, and the AI Analytics & Decision Support components shown in the TO-BE diagram.

Summary of Deliverables

Deliverable	Level	Format	Description
Deliverable 1	Level 1: Context Diagram	Diagram + Documentation	System boundaries, users, external systems
Deliverable 2	Level 2: Container Diagram	Diagram + Documentation	Technology containers, communication protocols
Deliverable 3	Level 3: Component Diagram	Diagram + Documentation	Internal structure of API container
Deliverable 4	Level 4: Code (Optional)	UML/ER Diagrams	Detailed implementation of critical components
Deliverable 5	Architecture Decision Records (ADRs)	Document	Key design decisions and rationale

Deliverable	Level	Format	Description
Deliverable 6	Technology Stack Matrix	Spreadsheet	Selected technologies with justifications
Deliverable 7	Traceability Matrix	Spreadsheet	Links to pain points, processes, data architecture
Deliverable 8	Executive Presentation	Slides	Summary for stakeholders

Deliverable 1: Level 1 - Context Diagram

Format

- Professional diagram (Lucidchart, [Draw.io](#), Structurizr, or similar)
- Accompanying documentation (2-3 pages)
- Filename: `TeamName_C4_Context.pdf` and `TeamName_C4_Context_Source`

Description

The Context Diagram shows the system boundaries and its relationships with external entities. It answers: What are we building? Who uses it? What other systems does it interact with? This is the "30,000-foot view" that everyone in the organization should understand.

Requirements

1. System Definition

Define the central system being modeled. Based on the TO-BE diagram, this is the **Unified Integration & Intelligence Layer** that encompasses:

- IT Asset Management System (Snipe-IT core)
- Polyglot Persistence Layer
- AI Analytics & Decision Support

2. Actors (Users)

Identify all user types who interact with the system:

Actor	Description	Interactions
Employee	Faculty, staff, students who request/use assets	Request equipment, view assigned assets, report issues
Manager	Department heads who approve requests	Approve/reject requests, view team assets
IT Help Desk	Tier 1 support staff	Check-in/out assets, update status, basic troubleshooting
IT Asset Manager	Specialized asset management team	Configure system, manage lifecycle, generate reports
Property Office	Property and Supply Office staff	Oversee asset accountability, approve disposals
Network Administrator	IT staff managing network infrastructure	View network relationships, configure auto-discovery

Finance Staff	Budget and accounting personnel	View asset values, depreciation, cost center allocations
----------------------	---------------------------------	--

3. External Systems

Identify all external systems that interact with the platform:

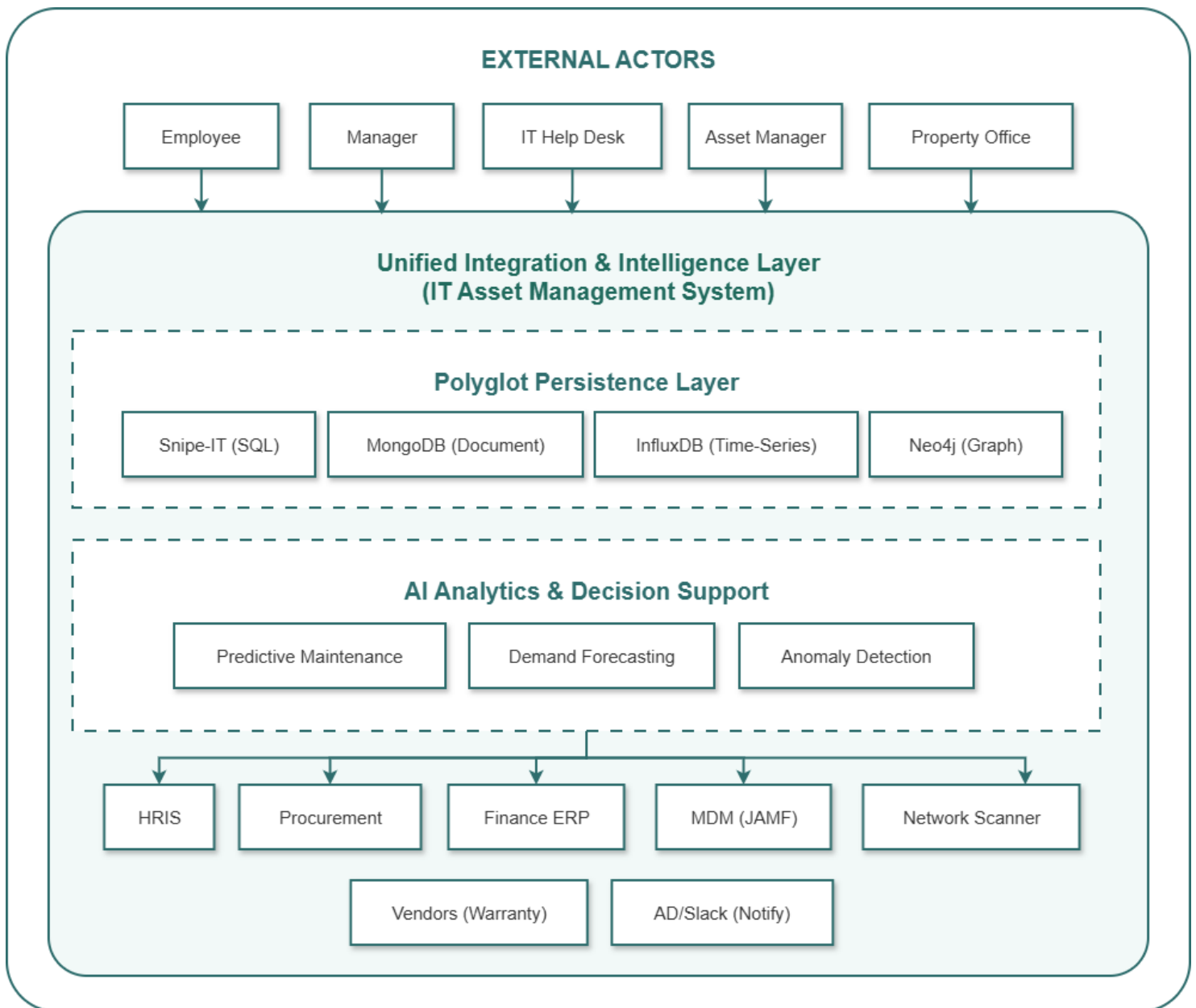
External System	Type	Data Exchange	Direction
HRIS (Human Resources)	HR System	Employee data (status, department, location)	Inbound only
Procurement System	ERP	Purchase orders, vendor data, costs	Inbound only
Finance System (ERP)	Financial	Depreciation data, cost allocations	Outbound only
MDM (JAMF/Intune)	Device Management	Telemetry data, compliance status	Bidirectional
Network Scanner	Infrastructure	Asset-to-network mappings, switch ports	Inbound only
Vendor Portals	External	Warranty claims, repair status	Bidirectional
Email/Slack	Communication	Notifications, alerts	Outbound only
Active Directory	Identity	Authentication, user groups	Inbound only

4. Data Flows

Document key data flows between the system and external entities:

Flow Name	Source	Destination	Content	Direction
Employee Data	HRIS	System	Employee ID, Name, Department, Status, Location	Inbound
Termination Event	HRIS	System	Employee ID, Termination Date, Last Working Day	Inbound
PO Data	Procurement	System	PO Number, Vendor, Items, Costs, ETA	Inbound
Asset Depreciation	System	Finance	Asset ID, Cost, Age, Current Value	Outbound
Telemetry Data	MDM	System	CPU Temp, Battery Health, Disk Usage, Location	Inbound
Health Alerts	System	IT Staff	Asset ID, Failure Probability, Recommendation	Outbound
Network Map	Network Scanner	System	Asset ID, Switch Port, AP, VLAN, Last Seen	Inbound
Relationship Data	System	Neo4j	Asset-to-network connections	Internal

5. Context Diagram Example



Deliverable 2: Level 2 - Container Diagram

Format

- Professional diagram (Lucidchart, [Draw.io](https://draw.io), Structurizr, or similar)
- Accompanying documentation with container descriptions (3-4 pages)
- Filename: `TeamName_C4_Container.pdf` and `TeamName_C4_Container_Source`

Description

The Container Diagram zooms into the system to show the high-level technology building blocks. A "container" represents a deployable unit—something that runs independently, such as a web server, mobile app, database, or message broker. This level answers: What are the major technical components? How do they communicate? What technologies are we using?

Requirements

1. Container Identification

Identify all deployable containers in the system:

Container	Type	Technology	Purpose
Web Application	SPA	React.js	Admin interface for IT staff, managers, property office
Mobile Application	Mobile	React Native	Field operations for scanning, check-in/out

Advanced System Design and Implementation

Container	Type	Technology	Purpose
API Gateway	Service	Kong / AWS API Gateway	Central entry point, authentication, rate limiting
API Application	Service	Python/Flask or Node.js	Business logic, orchestration, REST/GraphQL endpoints
Asset Synchronization Service	Service	Python/Java	CDC from Snipe-IT to other stores
Telemetry Ingestion Service	Service	Python/Go	Real-time ingestion of health data from MDM
Predictive Analytics Service	Service	Python	AI/ML model serving for predictions
Workflow Orchestrator	Service	Temporal	Workflow management for complex processes
PostgreSQL Database	Database	PostgreSQL 14+	Core asset records (Snipe-IT)
Document Store	Database	MongoDB	Unstructured metadata, warranty PDFs
Time-Series Database	Database	InfluxDB/TimescaleDB	Telemetry data (CPU, battery, location)
Graph Database	Database	Neo4j	Network relationships, department hierarchies

Container	Type	Technology	Purpose
Cache	Cache	Redis	Session storage, prediction caching, rate limiting
Message Broker	Queue	Kafka	Event distribution, CDC events
Search Engine	Search	Elasticsearch	Asset search, full-text indexing
Batch Processor	Batch	Airflow	Nightly reports, data exports

2. Container Communication

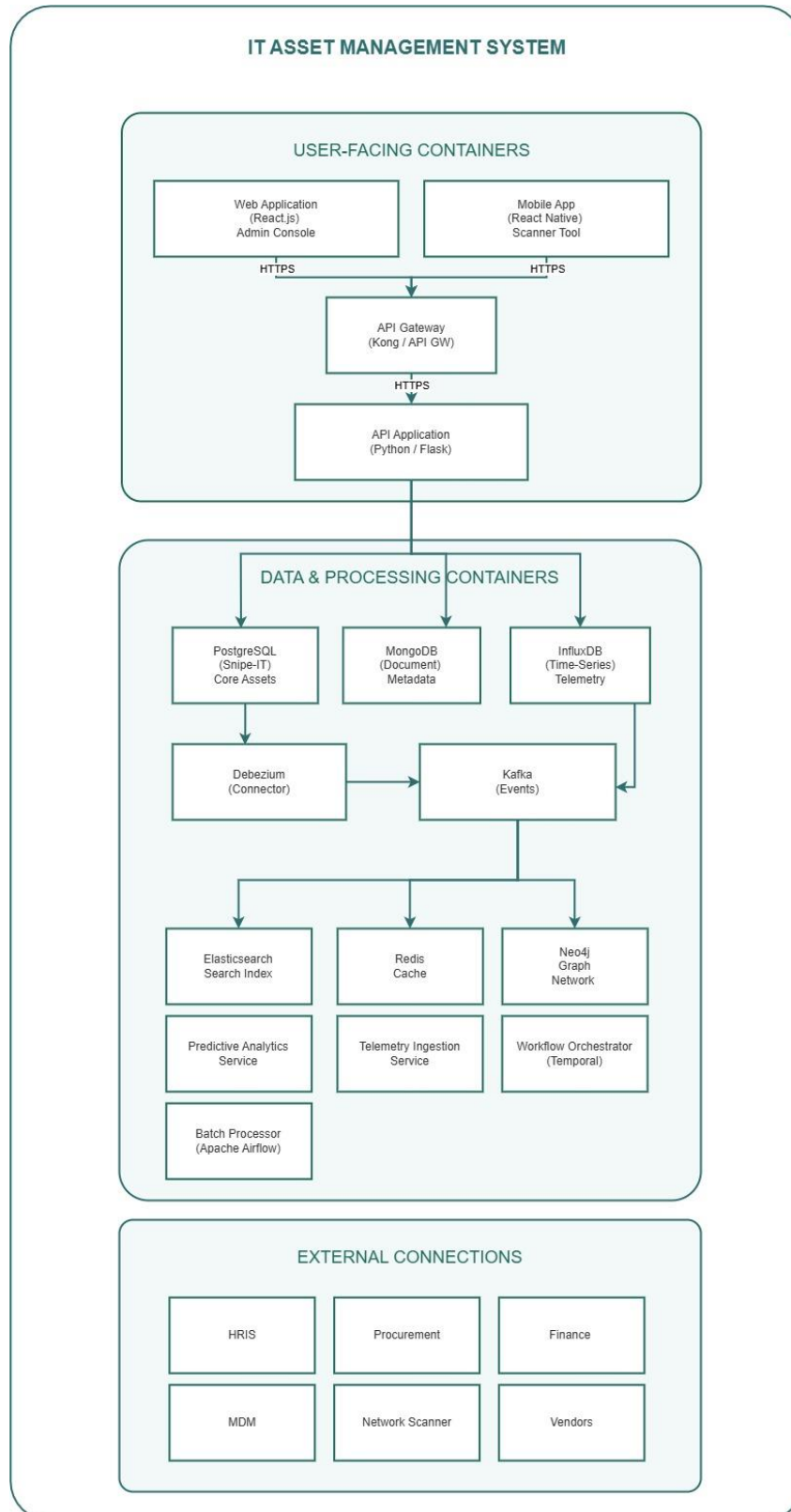
Document how containers communicate:

Source Container	Destination Container	Protocol	Purpose
Web App	API Gateway	HTTPS	All user interactions
Mobile App	API Gateway	HTTPS	Scanning, check-in/out
API Gateway	API Application	HTTPS	Authenticated requests
API Application	PostgreSQL	JDBC/ORM	CRUD operations
API Application	MongoDB	Native Driver	Product catalog, metadata
API Application	Redis	Redis Protocol	Session, caching

Advanced System Design and Implementation

Source Container	Destination Container	Protocol	Purpose
API Application	Kafka	Kafka Protocol	Event publishing
Asset Sync Service	PostgreSQL	CDC (Debezium)	Change capture
Asset Sync Service	Kafka	Kafka Protocol	Event publishing
Telemetry Service	MDM	HTTPS/REST	Pull telemetry data
Telemetry Service	InfluxDB	InfluxDB API	Store time-series data
Predictive Service	InfluxDB	InfluxDB API	Query historical data
Predictive Service	Kafka	Kafka Protocol	Consume events, publish alerts
Workflow Service	PostgreSQL	JDBC	State persistence
Batch Processor	PostgreSQL	JDBC	Query for exports

3. Container Diagram Example



4. Container Descriptions Table

Container	Technology	Purpose	Key Features	Deployment
Web Application	React.js	Admin interface	Dashboard, asset search, reports, approval workflows	S3 + CloudFront
Mobile App	React Native	Field operations	QR scanning, offline capability, location updates	App Store, Play Store
API Gateway	Kong	Entry point	Authentication, rate limiting, request routing	EKS/ECS
API Application	Python/Flask	Business logic	REST/GraphQL endpoints, orchestration	EKS/ECS (scalable)
PostgreSQL	PostgreSQL 14+	Core asset data	ACID compliance, complex queries	RDS Multi-AZ
MongoDB	MongoDB 6+	Unstructured metadata	Flexible schema for warranty docs, configs	MongoDB Atlas
InfluxDB	InfluxDB	Telemetry storage	Time-series optimization, high write throughput	InfluxDB Cloud

Container	Technology	Purpose	Key Features	Deployment
Neo4j	Neo4j	Network relationships	Graph traversal, relationship mapping	Neo4j Aura
Redis	Redis 7+	Caching	Session storage, prediction cache	ElastiCache
Kafka	Kafka	Event streaming	CDC events, notifications	MSK / Confluent
Elasticsearch	Elasticsearch 8+	Search	Full-text search, asset lookup	Elastic Cloud
Debezium	Debezium	CDC	PostgreSQL change capture	Kafka Connect
Predictive Service	Python/Flask	AI inference	Model serving, predictions	EKS/ECS
Telemetry Service	Python/Go	Data ingestion	MDM integration, telemetry processing	EKS/ECS
Workflow Engine	Temporal	Orchestration	Order fulfillment, offboarding	Temporal Cloud
Batch Processor	Airflow	Scheduled jobs	Reports, exports, syncs	MWAA / Airflow

Deliverable 3: Level 3 - Component Diagram

Format

- Professional diagram for at least one critical container (typically the API Application)
- Accompanying documentation (3-4 pages)
- Filename: TeamName_C4_Component.pdf and TeamName_C4_Component_Source

Description

The Component Diagram zooms into a specific container to show its internal structure. Components are groupings of related functionality within a container. This level answers: How is the API application structured? What are the major internal parts? How do they interact?

For this deliverable, students should select the **API Application** container (or another critical container) and decompose it into its internal components.

Requirements

1. Component Identification

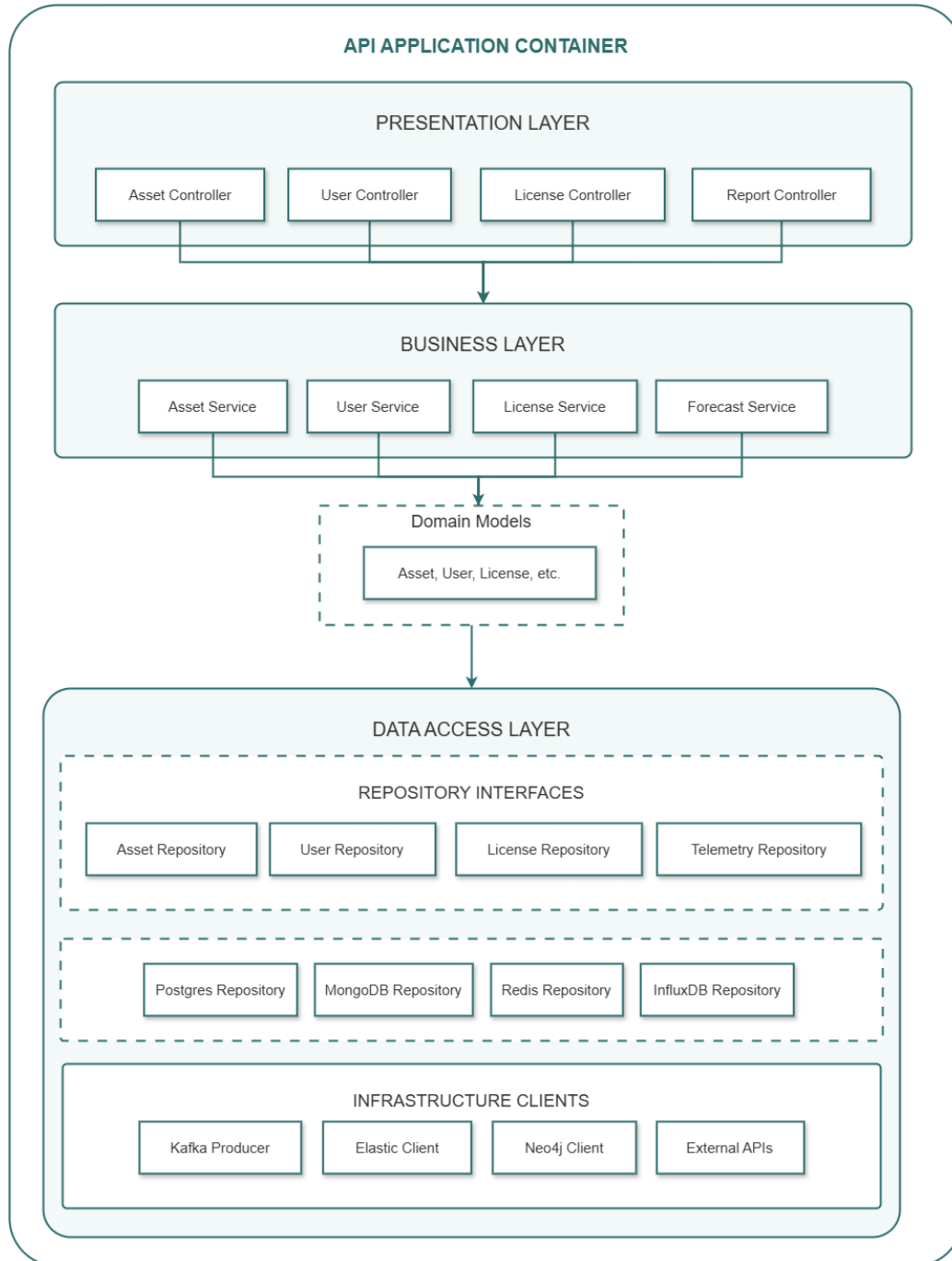
Identify all major components within the selected container:

Component	Type	Responsibility
Controllers	HTTP Layer	Handle requests, validation, response formatting
Services	Business Logic	Orchestration, business rules, coordination

Component	Type	Responsibility
Domain Models	Domain	Business entities, domain logic
Repositories	Data Access	Abstract data persistence
Event Publishers	Integration	Publish events to Kafka
External Clients	Integration	Call external services (HRIS, MDM, etc.)
Cache Manager	Infrastructure	Redis cache operations
Search Client	Infrastructure	Elasticsearch operations

2. Layered Architecture

Organize components into logical layers:



3. Component Descriptions

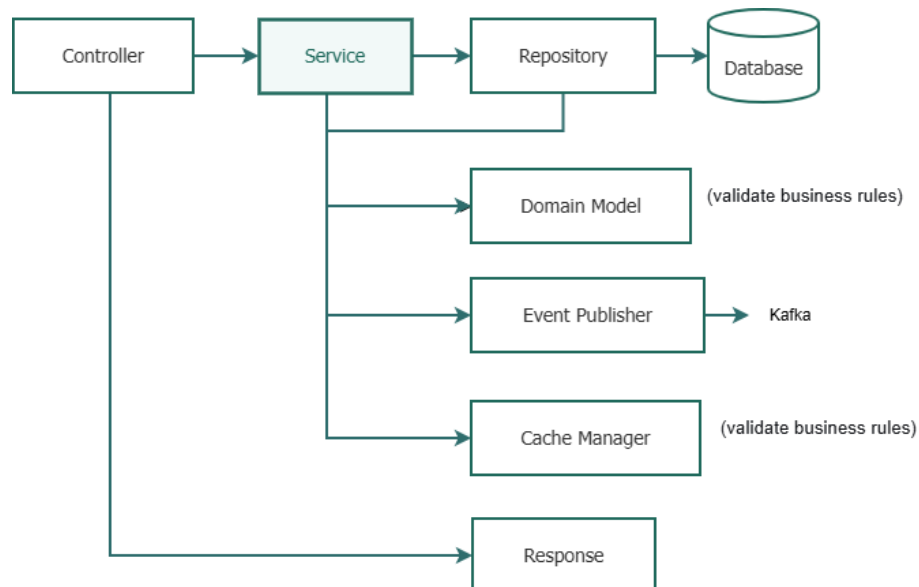
Component	Inputs	Outputs	Key Responsibilities	Dependencies
Asset Controller	HTTP Request	HTTP Response	Validate input, route to service, format response	Asset Service
Asset Service	Domain objects	Domain objects	Business logic, rule enforcement, coordination	Asset Repo, User Repo, Event Publisher
Asset Repository (Interface)	Query params	Domain objects	Abstraction for data access	None (interface)
Postgres Asset Repository	Query params	Domain objects	PostgreSQL CRUD operations	SQLAlchemy, Postgres DB
MongoDB Asset Repository	Query params	Domain objects	Metadata operations	MongoDB Client
Domain Models	Service calls	Domain objects	Encapsulate business behavior	None
Event Publisher	Domain events	Kafka messages	Publish change events	Kafka Producer
Cache Manager	Cache keys	Cached values	Redis operations	Redis Client

Component	Inputs	Outputs	Key Responsibilities	Dependencies
Search Client	Search queries	Search results	Elasticsearch queries	Elasticsearch Client

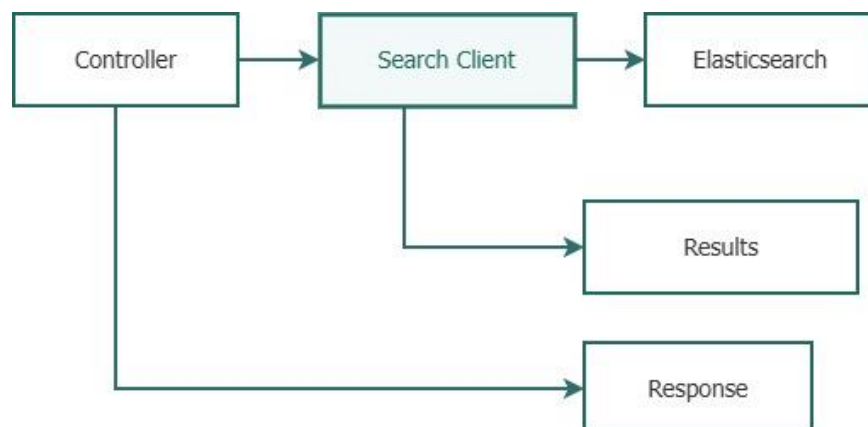
4. Component Interaction Flows

Document key interaction flows:

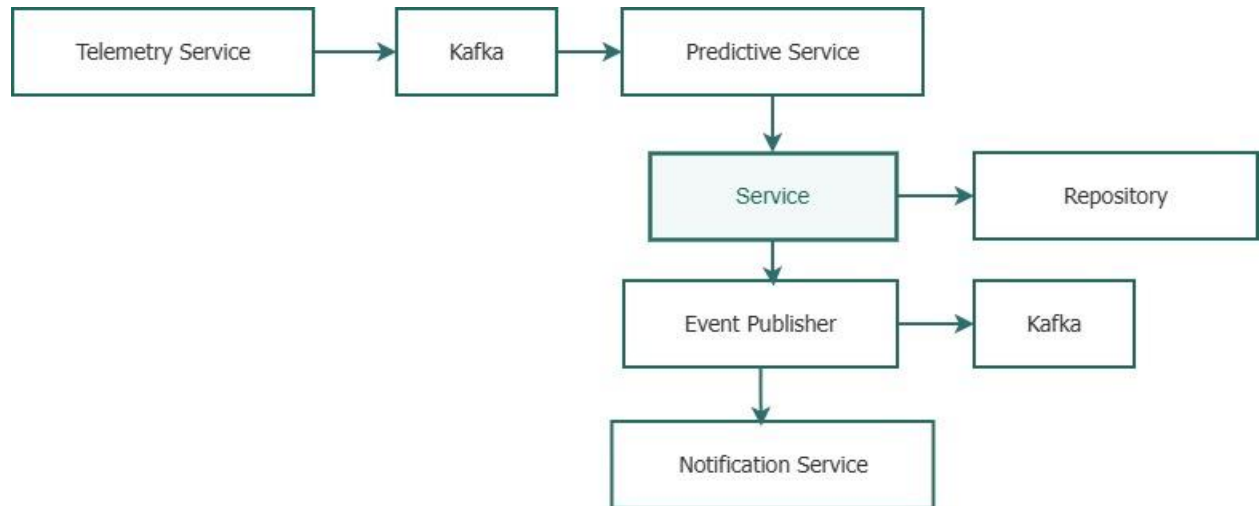
Flow 1: Asset Assignment Flow



2: Asset Search



Flow 3: Predictive Maintenance Alert



Deliverable 4: Level 4 - Code (Optional/Introductory)

Format

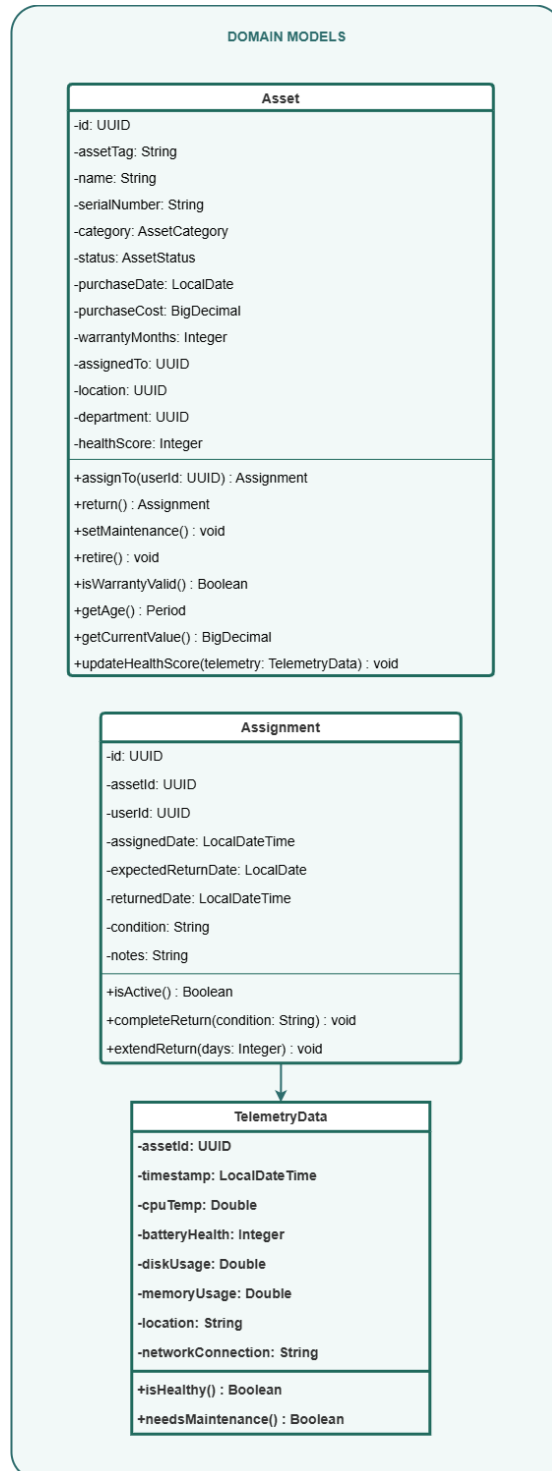
- UML Class Diagrams or Entity-Relationship Diagrams
- For critical components only (2-3 diagrams)
- Filename: TeamName_C4_Code.pdf

Description

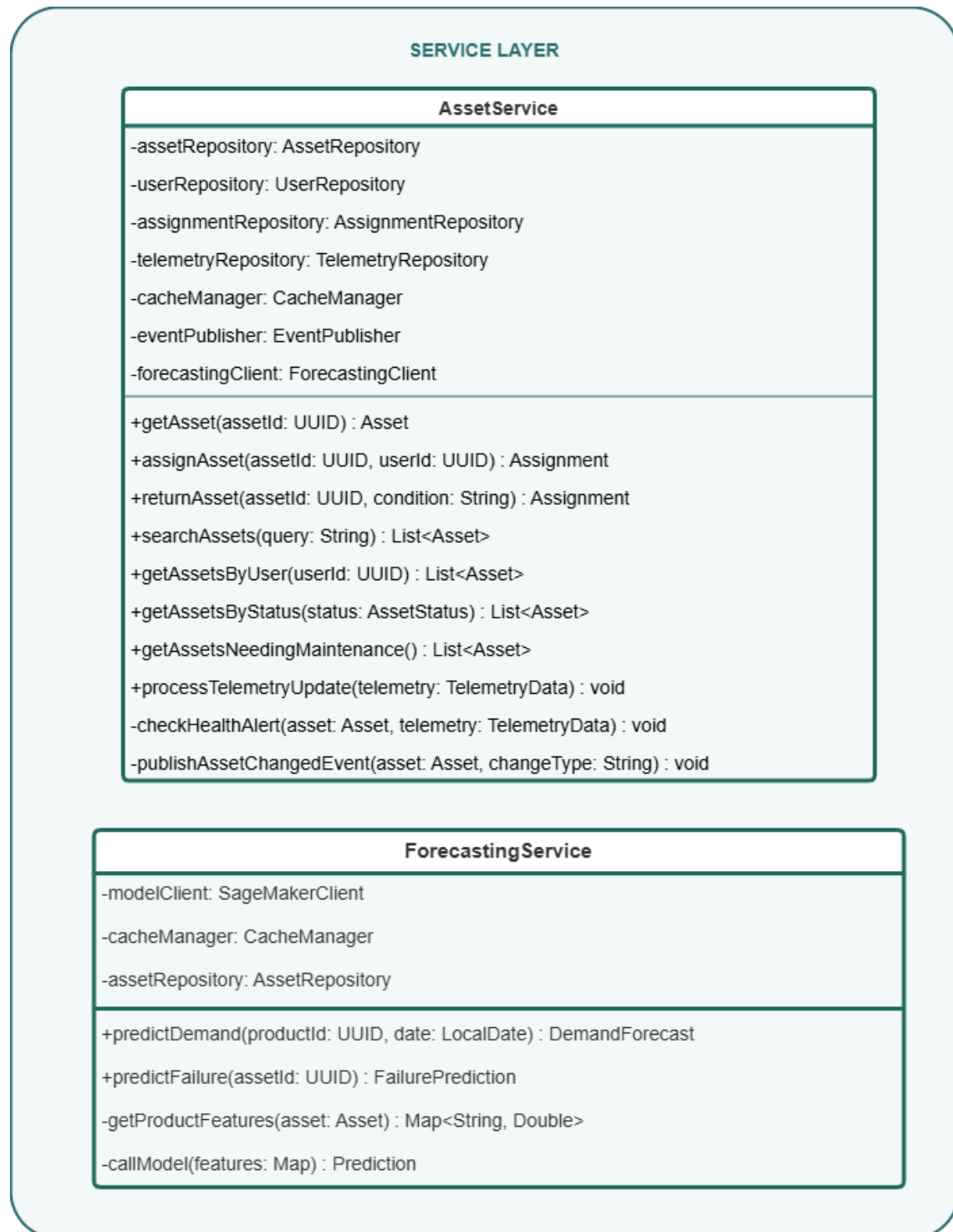
The Code level shows detailed implementation of critical components. This level is typically only created for complex or critical components that need special attention. It answers: How is this specific component implemented? What are its classes, methods, and relationships?

Requirements

1. Asset Entity Class Diagram



2. Asset Service Class Diagram



Deliverable 5: Architecture Decision Records (ADRs)

Format

- Document with individual ADR entries
- Filename: TeamName_ADRs.pdf

Description

Architecture Decision Records capture key design decisions, their context, rationale, and consequences. This ensures that future team members understand why the architecture was designed a certain way.

Requirements

For at least 5 key decisions, document ADRs using this template:

Field	Description
Title	Short, descriptive name of the decision
Status	Proposed, Accepted, Deprecated, Superseded
Context	The problem or situation requiring a decision
Decision	The chosen approach
Alternatives Considered	What else was considered and why not chosen
Consequences	Positive and negative implications
Related Decisions	Links to other ADRs

Date	When decision was made
------	------------------------

Example ADRs

ADR-001: Polyglot Persistence Strategy

Field	Value
Title	Use Polyglot Persistence with Multiple Database Technologies
Status	Accepted
Context	The system must handle diverse data types: core asset records (relational), unstructured metadata (documents), telemetry (time-series), and network relationships (graph). A single database cannot optimally serve all these use cases.
Decision	Adopt polyglot persistence using: PostgreSQL (Snipe-IT) for core asset records, MongoDB for metadata, InfluxDB for telemetry, Neo4j for network relationships. Use Debezium for CDC synchronization.
Alternatives	Single relational database (rejected: poor performance for telemetry, inflexible for metadata). Single document database (rejected: weak ACID for transactions).
Consequences	Positive: optimal performance for each use case, flexible schemas. Negative: increased operational complexity, eventual consistency challenges, need for CDC infrastructure.
Related Decisions	ADR-002 (Event-Driven Synchronization), ADR-003 (CDC Strategy)
Date	2025-03-15

ADR-002: AI Integration Pattern

Field	Value
Title	Use Event-Driven Pattern for Predictive Maintenance
Status	Accepted
Context	Predictive maintenance requires analyzing telemetry data from thousands of assets. Real-time analysis is not required; predictions can be generated asynchronously.
Decision	Implement event-driven AI using Kafka. Telemetry data published to Kafka; Predictive Analytics Service consumes events, runs models, and publishes alerts to Kafka for downstream processing.
Alternatives	Request-reply pattern (rejected: would require synchronous calls for every telemetry update, creating high load). Batch processing only (rejected: would miss real-time anomaly detection).
Consequences	Positive: decoupled, resilient, scalable. Negative: eventual consistency in alerts, additional infrastructure complexity.
Related Decisions	ADR-001 (Polyglot Persistence)
Date	2025-03-16

Deliverable 6: Technology Stack Matrix

Format

- Spreadsheet or table in document
- Filename: TeamName_Technology_Stack.xlsx

Description

The Technology Stack Matrix documents all technology choices made in the application architecture, with justifications for each selection.

Requirements

Component	Technology	Version	Justification	Alternatives Considered
Web Frontend	React.js	18.x	Strong ecosystem, team expertise, component reusability	Angular (rejected: steeper learning curve), Vue (rejected: smaller ecosystem)
Mobile App	React Native	0.72	Code reuse with web, large community, good for scanning apps	Flutter (rejected: requires learning Dart), Native (rejected: no code reuse)
API Application	Python/Flask	2.3	Rapid development, excellent ML/AI	Node.js (rejected: weaker ML ecosystem),

Advanced System Design and Implementation

			libraries, team skills	Java/Spring (rejected: slower development)
API Gateway	Kong	3.x	Open source, plugin ecosystem, rate limiting, authentication	AWS API Gateway (rejected: vendor lock-in), NGINX (rejected: less built-in features)
Core Database	PostgreSQL	14+	ACID compliance, JSON support, mature, open source	MySQL (rejected: weaker JSON support), SQL Server (rejected: licensing cost)
Document Store	MongoDB	6+	Flexible schema, good for metadata, easy scaling	Couchbase (rejected: less adoption), DynamoDB (rejected: vendor lock-in)
Time-Series	InfluxDB	2.x	Optimized for telemetry, high write throughput	TimescaleDB (rejected: PostgreSQL-based but higher overhead)
Graph Database	Neo4j	5.x	Best for relationship queries, mature driver support	Amazon Neptune (rejected: vendor lock-in), ArangoDB (rejected: less mature)

Advanced System Design and Implementation

Cache	Redis	7.x	High performance, data structures, widely adopted	Memcached (rejected: fewer data structures)
Message Broker	Kafka	3.x	High throughput, durability, ecosystem	RabbitMQ (rejected: lower throughput), AWS SQS (rejected: vendor lock-in)
CDC	Debezium	2.x	Native PostgreSQL support, Kafka integration	Custom CDC (rejected: re-inventing wheel)
Workflow Engine	Temporal	1.20	Stateful workflows, durable execution, retries	Airflow (rejected: batch-oriented), Camunda (rejected: Java-centric)
Batch Processing	Airflow	2.x	Mature DAG management, scheduling	Prefect (rejected: newer, less mature)
Search	Elasticsearch	8.x	Full-text search, faceted search, analytics	OpenSearch (rejected: forked, less ecosystem)
AI Model Serving	SageMaker	-	Integrated with AWS, managed endpoint, monitoring	Custom container (rejected: more operational overhead)

Advanced System Design and Implementation

Container Orchestration	Kubernetes	1.28	Portability, scaling, ecosystem	ECS (rejected: AWS-specific)
Infrastructure as Code	Terraform	1.5	Cloud-agnostic, mature, state management	Pulumi (rejected: less mature, smaller community)
CI/CD	GitLab CI	-	Integrated with source control, flexible	Jenkins (rejected: maintenance overhead), GitHub Actions (rejected: external repo)
Monitoring	Prometheus + Grafana	-	Industry standard, flexible, open source	Datadog (rejected: cost), CloudWatch (rejected: vendor lock-in)
Logging	ELK Stack	8.x	Centralized logging, search, visualization	Splunk (rejected: cost), Loki (rejected: less mature)
Tracing	Jaeger	1.50	OpenTelemetry compatible, distributed tracing	Zipkin (rejected: less features)

Deliverable 7: Traceability Matrix

Format

- Spreadsheet
- Filename: TeamName_App_Traceability.xlsx

Description

The Traceability Matrix links your application architecture decisions back to the pain points from Activity 1, the TO-BE processes from Activity 2, and the data architecture from Activity 3. This ensures that every architectural decision is justified by business need.

Requirements

1. Pain Point to Application Solution Traceability

Pain Point ID	Pain Point Description	Application Solution	C4 Level	Component
P1	Procurement-to-inventory delay	Procurement API integration; automated asset creation	Container	API Application, Kafka
P3	Poor mobile experience	React Native mobile app with offline scanning	Container	Mobile Application

Advanced System Design and Implementation

Pain Point ID	Pain Point Description	Application Solution	C4 Level	Component
P5	License over-purchasing	License Management Service with usage tracking	Component	License Service
P7	Asset non-return after offboarding	HRIS integration; Temporal workflow for offboarding	Container	Workflow Orchestrator
P9	Finance spreadsheets	ERP API integration; automated depreciation export	Component	Batch Processor
P12	Audit findings	Graph database for relationships; assignment history	Container	Neo4j, Assignment History
P15	Predictive failures	Telemetry ingestion; Predictive Analytics Service	Container	Telemetry Service, Predictive Service

2. TO-BE Process to Application Support Traceability

TO-BE Process	Application Support	Containers Involved	Components
1.0 User Sync	HRIS webhook receiver	API Application	User Controller, User Service, Kafka Producer
2.0 Procurement Intake	Procurement API integration	API Application, Kafka	Procurement Controller, Asset Service
3.0 Asset Assignment	Assignment workflow	API Application, Temporal	Asset Service, Workflow Client
4.0 Telemetry Ingestion	MDM integration, telemetry processing	Telemetry Service, InfluxDB	Telemetry Consumer, InfluxDB Repository
5.0 AI Analytics	Predictive models	Predictive Service, Kafka	Forecasting Service, Anomaly Detection
6.0 License Management	License allocation, compliance	API Application, PostgreSQL	License Service, License Repository
7.0 Asset Retirement	Retirement workflow	Temporal, API Application	Retirement Workflow, Asset Service

3. Data Architecture to Application Mapping

Data Architecture Element	Application Support	Container	Component
Enterprise Asset Entity	Asset domain model	API Application	Asset Entity, Asset Service
Snipe-IT Core Tables	Asset repository implementation	API Application	Postgres Asset Repository
Custom Assignment History	Assignment tracking	API Application, PostgreSQL	Assignment Repository, Audit Service
Telemetry Time-Series	Telemetry ingestion and query	Telemetry Service, Predictive Service	InfluxDB Repository, Forecasting Service
Graph Network Relationships	Network mapping queries	API Application	Neo4j Client, Network Service
CDC Event Stream	Change propagation	Debezium, Kafka	Kafka Producer, Sync Consumers

Deliverable 8: Executive Presentation

Format

- 8-10 slides, professional design
- Filename: TeamName_App_Architecture_Presentation.pptx

Description

A concise presentation summarizing your application architecture for executive stakeholders.

Slide Content

Slide	Content
1: Title	Project title, team members, date
2: Executive Summary	Overview of application architecture transformation, key outcomes
3: C4 Level 1 - Context	System boundaries and external interactions
4: C4 Level 2 - Containers	Major technology containers and communication
5: C4 Level 3 - Components	Internal structure of API application
6: Key Technology Decisions	Top 5 technology choices with justifications
7: AI Integration Approach	How AI is integrated (predictive maintenance, forecasting)

8: Deployment Architecture	How containers map to infrastructure (cloud/on-prem)
9: Traceability to Business Goals	How architecture addresses pain points
10: Next Steps	Implementation roadmap, Q&A

Evaluation Rubric

Criterion	Weight	Excellent (90-100%)	Satisfactory (70-89%)	Needs Improvement (<70%)
Level 1 Context Diagram	15%	Clear, complete, professional; all actors and systems identified	Good; most actors identified	Missing key actors or systems
Level 2 Container Diagram	20%	All containers identified; clear technology choices; communication protocols documented	Most containers identified; technology choices reasonable	Missing containers or unclear communication
Level 3 Component Diagram	20%	Well-structured layers; clear component responsibilities;	Good structure; most	Unclear layering or missing interactions

Criterion	Weight	Excellent (90-100%)	Satisfactory (70-89%)	Needs Improvement (<70%)
		interaction flows documented	components identified	
Architecture Decision Records	10%	5+ ADRs with clear context, rationale, consequences	3-4 ADRs with reasonable justification	Few ADRs or poor justification
Technology Stack Matrix	10%	Comprehensive, justified selections	Good coverage, reasonable justifications	Missing or unjustified selections
Traceability Matrix	10%	Clear links to pain points, processes, data architecture	Some traceability established	Missing or unclear traceability
Executive Presentation	10%	Professional, compelling, tells coherent story	Good presentation, minor issues	Poor organization or unclear
Overall Coherence	5%	All deliverables consistent, aligned with business goals	Generally consistent	Inconsistent across deliverables

Key Success Factors

1. **Start with the TO-BE Business Architecture:** Your application architecture must enable the TO-BE processes. Use the TO-BE BPMN diagrams as your requirements.
2. **Use the C4 Model Hierarchically:** Each level should logically decompose the previous level. The Container diagram should be a decomposition of the Context diagram; the Component diagram should decompose a specific container.
3. **Document Technology Choices:** Every technology selection should have a clear justification. "Because we like it" is not sufficient.
4. **Create Traceability:** Link every major decision back to pain points, processes, or data requirements. This prevents "solutioneering."
5. **Consider Deployment:** Your container diagram should be deployable. Think about how containers will run in cloud or on-premises environments.
6. **Balance Detail and Clarity:** The Component diagram should show enough detail to understand the structure but not every class. Save detailed class diagrams for Level 4.
7. **Use Standard Notation:** Follow C4 model conventions. Be consistent across diagrams.

Connection to Course Modules

Module	Connection
Module 2	C4 model introduced; this activity applies it to your project
Module 3	Polyglot persistence and AI integration patterns are implemented in containers

Advanced System Design and Implementation

Module 4	Development and implementation will follow the C4 structure
Module 5	Testing, deployment, and operations will be designed for these containers

End of Activity 4 Description